

Serialisierung und Datei-I/O – Teil 1

Serialisierung

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 26. Januar 2026 }

inno



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Serialisierung und Datei-I/O

 Plan für diese Woche

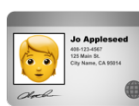
 Zustand speichern

 Serialisierung

 Interface Serializable

 Flüchtige Instanzvariablen

 Deserialisierung

 Versions-ID

 Textdatei schreiben

 Klasse File

 Puffer

 Textdatei lesen

 Try-With-Resources

 Pair Programming

 Spaß

📖 Serialisierung und Datei-I/O

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

🫙 Welche Möglichkeiten gibt es, den Zustand eines Programms zu speichern?

❄️ Was passiert mit einem Objekt, wenn es serialisiert wird?

❄️ Wozu dient das Interface `Serializable`?

💨 Was sind „flüchtige“ Instanzvariablen?

🔥 Was passiert bei der Deserialisierung?

📄 Wann sollte die Versions-ID einer Klasse geändert werden?

🙋 Welche Fragen haben Sie zu diesem Kapitel?

16 Serialisierung und Datei-I/O

Objekte (und Text) speichern



Objekte lassen sich flach drücken und wieder aufblasen. Objekte besitzen Zustand und Verhalten. Das *Verhalten* lebt in der *Klasse*, während sich der *Zustand* in den einzelnen Objekten befindet. Was passiert aber, wenn es Zeit ist, den Zustand eines Objekts zu speichern? Wenn Sie ein Spiel schreiben, benötigen Sie eine Möglichkeit, den Spielstand zu sichern und wiederherzustellen. Wenn Sie eine App für die Erstellung von Diagrammen programmieren, müssen Sie die Datei irgendwie speichern und wieder öffnen können. Soll Ihr Programm Zustände speichern, können Sie das entweder *auf die harte Tour machen*, indem Sie jedes Objekt befragen und dann mühsam die Werte aller Instanzvariablen in eine Datei schreiben, die ein von Ihnen entworfenes Format hat. Oder **Sie gehen einfach nach OO-Art vor**. Hierfür wird das Objekt selbst gefriergetrocknet/flach gedrückt/persistent gemacht/dehydriert. Um es zurückzubekommen, wird es wieder aufgetaut/aufgeblasen/wiederhergestellt/rehydriert. Trotz dieser Möglichkeiten kann es *manchmal* sein, dass es nur auf die harte Tour geht, besonders wenn die Datei, die Ihre App speichern muss, von einer Nicht-Java-Applikation gelesen werden muss. Daher sehen wir uns in diesem Kapitel beide Wege an. Und weil alle I/O-Operationen (I/O = E/A, Ein-/Ausgabe) potenziell gefährlich sind, werfen wir auch gleich einen Blick auf bessere Möglichkeiten des Exception-Handlings.



Zustand speichern

2 Möglichkeiten

Zustand speichern

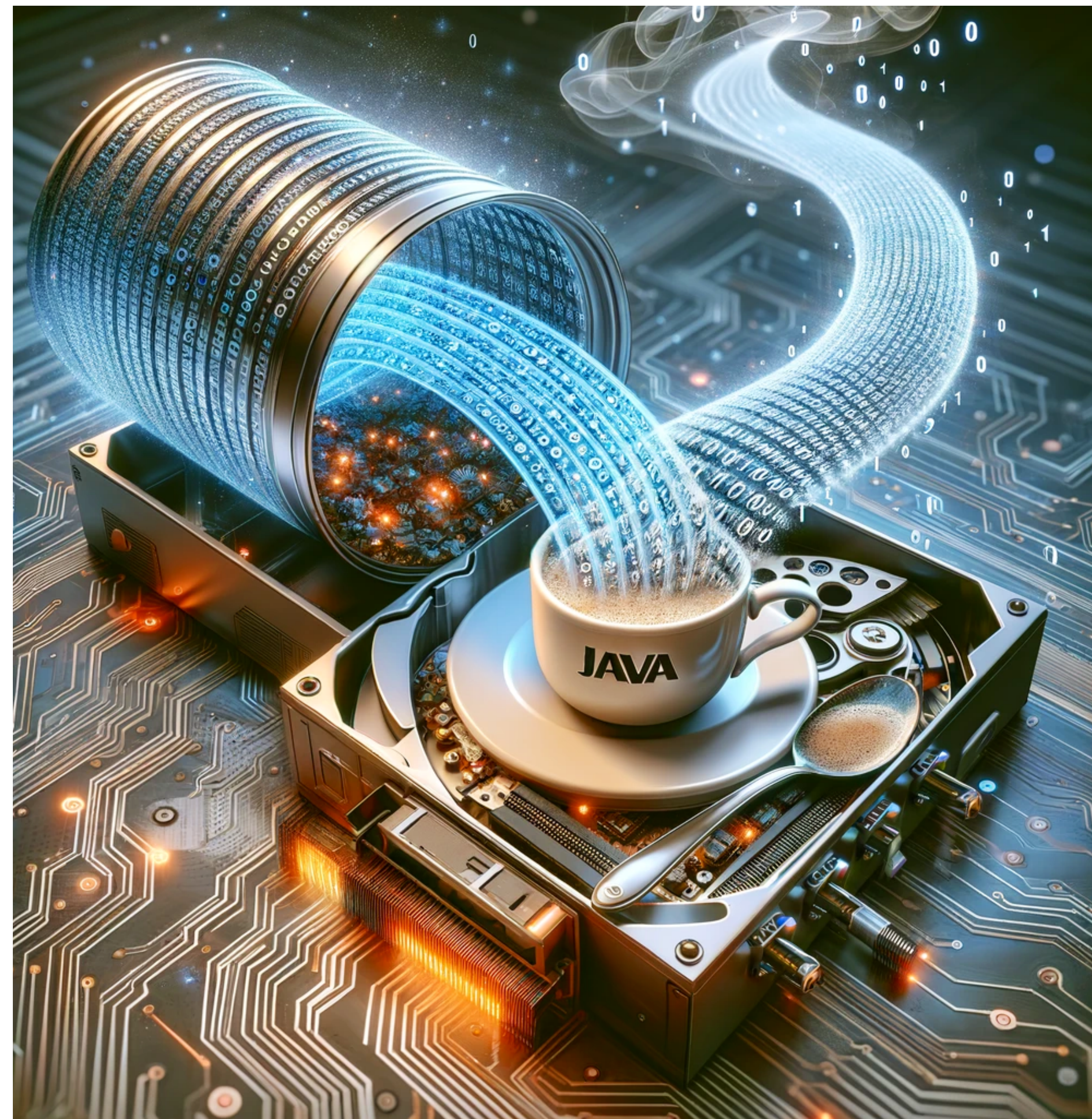
- Es gibt unter anderem zwei Möglichkeiten, den Zustand eines Objekts zum Beispiel in einer Datei zu speichern:
 1. Benutzen Sie **Serialisierung**.
 2. Schreiben Sie die Daten in eine **einfache Textdatei**.



Serialisierung

Zustand speichern

- Verwenden Sie Serialisierung, wenn Ihre Daten nur von dem Java-Programm verwendet werden, das sie erzeugt hat.
- Dabei wird eine Datei erzeugt, die die „flachgedrückten“ (serialisierten) Objekte enthält.
- Später kann Ihr Programm diese aus der Datei lesen und als wieder „aufgeblasene“ Objekte in den Heap packen.
- Die in einer solchen Datei enthaltenen Daten sind für den Menschen nur sehr schwer zu lesen.





Textdatei



Zustand speichern

- Wenn Ihre Daten von anderen Programmen verwendet werden sollen, schreiben Sie sie in eine einfache Textdatei.
- Dort können Sie die Daten zum Beispiel durch Kommata oder Tabulatoren trennen.
- Solche Dateien können unter anderem von einer Tabellenkalkulation eingelesen werden.
- Diese Daten sind für Menschen viel leichter lesbar.



```
[
  {
    "name": "Buddy",
    "isHungry": true
  },
  {
    "name": "Max",
    "isHungry": false
  },
  {
    "name": "Bella",
    "isHungry": true
  }
]
```

JSON (JavaScript
Object Notation)

Object
Serialization
Stream Protocol

address	00 01 02 03 04 05 06 07	08 09 10 11 12 13 14 15
00000000	ac ed 00 05 73 72 00 03	44 6f 67 40 53 41 df 24
00000010	00 16 4b 02 00 02 5a 00	08 69 73 48 75 6e 67 72
00000020	79 4c 00 04 6e 61 6d 65	74 00 12 4c 6a 61 76 61
00000030	2f 6c 61 6e 67 2f 53 74	72 69 6e 67 3b 78 70 01
00000040	74 00 04 46 69 64 6f 73	71 00 7e 00 00 00 74 00
00000050	03 52 65 78 73 71 00 7e	00 00 01 74 00 04 53 70
00000060	6f 74	

08 09 10 11 12 13 14 15

UTF8 ☐ unsigned ☐ bigendian

....sr..Dog@SA.\$
..K...Z..isHungr
yL..namet..Ljava
/lang/String;xp.
t..Fidosq.~...t.
.Rexsq.~...t..Sp
ot

! Quiz

🫙 Zustand speichern

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Sie können den Zustand eines Objekts
durch dessen Serialisierung speichern.

Richtig!

Bei der Serialisierung werden Objekte
„flachgedrückt“ und in einem eher schwer
lesbaren Format gespeichert.

Richtig!

Die einzige Möglichkeit in Java, den Zustand zu speichern, ist Serialisierung.

Sie können die Daten zum Beispiel in eine Textdatei schreiben.

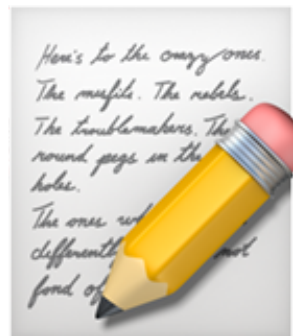


Welcher **eine** Gedanke zu



„Zustand speichern“ war

für Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren

Serialisierung

```
1 import java.io.FileOutputStream;
2 import java.io.IOException;
3 import java.io.ObjectOutputStream;
4 import java.time.LocalDate;
5
6 public class ObjectSerializationDemo {
7
8     // We have no exception handling here because this is just a demo.
9     // In real code, we would do exception handling.
10    public static void main(String[] args) throws IOException {
11        // FileOutputStream knows how to write bytes to a file.
12        // It creates a file if it does not exist. Warning: Otherwise it will be overwritten!
13        FileOutputStream fileStream = new FileOutputStream("objects.ser");
14
15        // ObjectOutputStream knows how to write objects to a stream.
16        ObjectOutputStream objectStream = new ObjectOutputStream(fileStream);
17
18        // Write the objects to the stream, i.e. to the file.
19        // The objects are written in the order they are written to the stream.
20        objectStream.writeObject("The current date is:");
21        objectStream.writeObject(LocalDate.now());
22        objectStream.writeObject(10);
23        objectStream.writeObject(47.11);
24
25        // Close the stream. This will flush the stream and close the file.
26        objectStream.close();
27    }
28 }
29
```

STDIN

Input for the program (Optional)

Output:

Program did not output anything!



Auf dem eigenen
Computer ausführen!

Anschluss- und Verkettungs-Streams

Serialisierung

In der Java-I/O-API gibt es zwei Arten von Streams:

- **Anschluss-Streams** (Connection Streams), die Verbindungen zwischen Quelle und Ziel (Dateien, Netzwerk-Sockets und so weiter) darstellen.
- **Verkettungs-Streams** (Chain Streams), die nur funktionieren, wenn sie mit anderen Streams „verkettet“ sind.

Häufig sind mindestens zwei Streams miteinander verbunden, um nützlich zu sein:

- Ein Stream stellt die Verbindung (den Anschluss) dar, während auf dem anderen Stream Methoden aufgerufen werden können.
- Die Anschluss-Streams arbeiten oft auf einer niedrigeren Ebene. Zum Beispiel hat `FileOutputStream` Methoden zum Schreiben von Bytes.
- Der Verkettungs-Stream `ObjectOutputStream` schreibt Objekte und arbeitet daher auf einer höheren Ebene.

Spezialisierte Streams

Serialisierung

- In der Regel ist jeder Stream in Java auf eine bestimmte Aufgabe spezialisiert.
- Zum Beispiel:
 - Schreiben / Lesen von Dateien, sowohl in binärer als auch in Textform.
 - Schreiben / Lesen von Text auf die / von der Konsole.
 - Schreiben / Lesen von Daten in Netzwerkkommunikation.
 - Verschlüsseln / Entschlüsseln von Daten.

Kombination von Streams

Serialisierung

- Die Möglichkeit, verschiedene Anschluss- und Verkettungs-Streams miteinander zu kombinieren, bietet eine enorme Flexibilität.
- Zum Beispiel:
 1. Lesen aus Textdatei,
 2. dann Daten verschlüsseln,
 3. dann über das Internet versenden,
 4. dann aus dem Internet empfangen,
 5. dann Daten entschlüsseln
 6. und schließlich Ausgabe als Text auf Konsole.

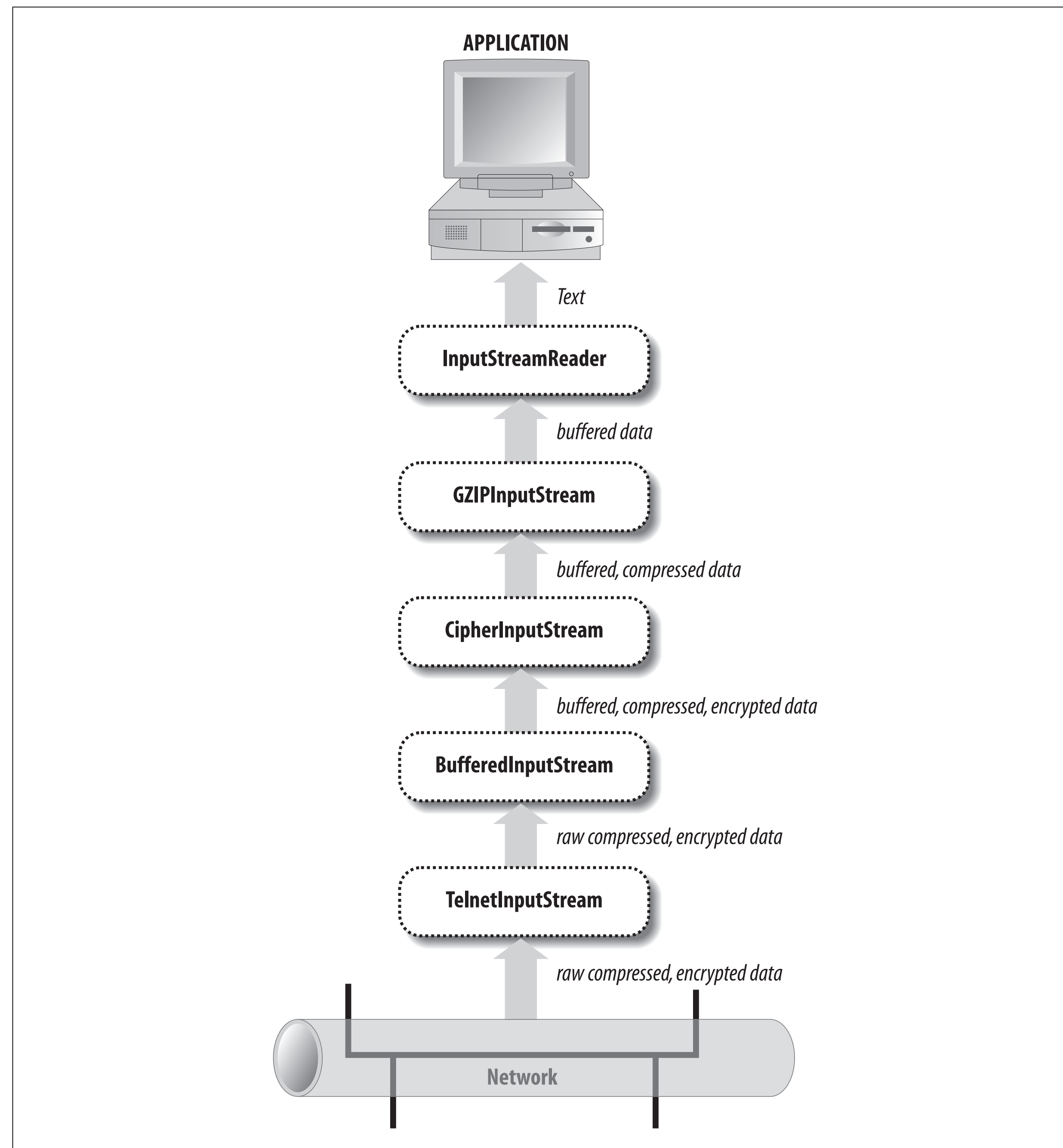


Figure 4-2. The flow of data through a chain of filters

**Jetzt Seiten 544
bis 546
wiederholen!**



Sie haben dafür 7 Minuten Zeit.

! ? Quiz

❄️ Serialisierung

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Sie benötigen einen `ObjectOutputStream`
(aus dem Package `java.io`), um ein Objekt
zu serialisieren.

Richtig!

Streams sind entweder Anschluss- oder Verkettungs-Streams.

Richtig!

Anschluss-Streams können eine Verbindung zu einer Quelle oder einem Ziel darstellen. Es muss sich um eine Datei handeln.

Typischerweise handelt es sich um eine Datei, eine Netzwerk-Socket-Verbindung oder eine Konsole.

Verkettungs-Streams können keine Verbindung zu einer Quelle oder einem Ziel herstellen und müssen mit einem Anschluss- (oder einem anderen) Stream verkettet werden.

Richtig!

Um ein Objekt in eine Datei zu serialisieren, erstellen Sie einen `FileOutputStream` und verketteten diesen mit einem `ObjectOutputStream`.

Richtig!

Um ein Objekt zu serialisieren, rufen Sie `writeObject(o)` auf dem `ObjectOutputStream` auf. Danach rufen Sie die gleichnamige Methode auf dem `FileOutputStream` auf.

Auf dem `FileOutputStream` müssen Sie keine Methoden aufrufen.

Bei der Serialisierung eines Objekts wird dessen gesamter Objektgraph serialisiert. Das bedeutet, dass alle Objekte, die von den Instanzvariablen des serialisierten Objekts referenziert werden, ebenfalls serialisiert werden, ebenso wie die Objekte, die von diesen Instanzvariablen referenziert werden, und so fort.

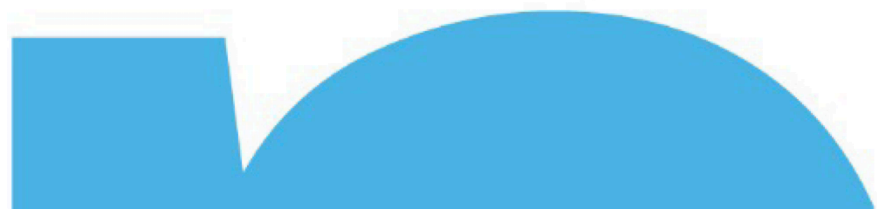
Richtig!

 Welcher **eine** Gedanke zu
„ Serialisierung“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren



Interface Serializable



Serialisierbare Klassen

🌨️ Interface `Serializable`

- Wenn Ihre Klasse serialisierbar sein soll, müssen Sie das `Serializable` Interface implementieren.
- Dieses Interface wird Marker- oder Tag-Interface genannt, da es keine implementierbaren Methoden enthält.
- Sein einziger Zweck besteht darin, die Klasse, die es implementiert, als serialisierbar zu deklarieren.
 - Objekte dieses Typs können mit Hilfe des Serialisierungsmechanismus gespeichert werden.
- Unterklassen einer serialisierbaren Superklasse sind ebenfalls serialisierbar.
 - Dies gilt auch dann, wenn die Unterklasse nicht explizit `implements Serializable` deklariert.



```
1 import java.io.FileOutputStream;
2 import java.io.IOException;
3 import java.io.ObjectOutputStream;
4
5 public class SerializableDemo {
6
7     // We have no exception handling here because this is just a demo.
8     // In real code, we would do exception handling.
9     public static void main(String[] args) throws IOException {
10         FileOutputStream fileStream = new FileOutputStream("/tmp/dog.ser");
11         ObjectOutputStream objectStream = new ObjectOutputStream(fileStream);
12
13         Dog dog = new Dog("Fido", true);
14         objectStream.writeObject(dog);
15
16         objectStream.close();
17     }
18 }
19
```

STDIN

Input for the program (Optional)

Output:

Exception in thread "main" java.io.NotSerializableException: DogCollar
at java.base/java.io.ObjectOutputStream.writeObject0(ObjectOutputStream.java:1085)
at java.base/java.io.ObjectOutputStream.defaultWriteFields(ObjectOutputStream.java:1451)
at java.base/java.io.ObjectOutputStream.writeSerialData(ObjectOutputStream.java:1408)
at java.base/java.io.ObjectOutputStream.writeOrdinaryObject(ObjectOutputStream.java:1317)
at java.base/java.io.ObjectOutputStream.writeObject0(ObjectOutputStream.java:1079)
at java.base/java.io.ObjectOutputStream.writeObject(ObjectOutputStream.java:325)
at SerializableDemo.main(SerializableDemo.java:14)

! ? Quiz

🧊 Interface Serializable

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Ein Objekt muss das Interface
`Serializable` implementieren, um
serialisierbar zu sein.

Richtig!

Wenn die Superklasse `Serializable` implementiert, ist die Unterklasse automatisch serialisierbar. Dies gilt auch dann, wenn die Unterklasse nicht explizit `implements Serializable` deklariert.

Richtig!

Ist ein Objekt im Graph nicht serialisierbar, wird zur Laufzeit eine Exception ausgelöst. Es sei denn, die Instanzvariable, die das Objekt referenziert, wird übersprungen (siehe nächstes Unterkapitel).

Richtig!

 Welcher **eine** Gedanke zu
„ Interface Serializable“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Flüchtige Instanzvariablen

Jetzt Seite 549 wiederholen!



Sie haben dafür 3 Minuten Zeit.

! ? Quiz

💨 Flüchtige Instanzvariablen

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**

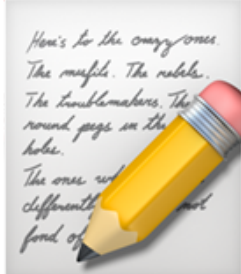


Wenn die Serialisierung ein Objekt überspringen soll, markieren Sie es mit dem Schlüsselwort `transient`.

Richtig!

Bei der Deserialisierung wird eine transiente Variable immer als `null` wiederhergestellt.

Nur für Objektreferenzen. Für elementare Typen als Standardwert.

☁️ Welcher **eine** Gedanke zu
„💧 Flüchtige Instanzvariablen“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

🧘 Durchatmen und 🧠 Sortieren

Deserialisierung

ObjectDeserializationDemo.javaDog.javathree-dogs.ser+

1import java.io.FileInputStream;

2import java.io.IOException;

3import java.io.ObjectInputStream;

4

5public class ObjectDeserializationDemo {

6

7// We have no exception handling here because this is just a demo.

8// In real code, we would do exception handling.

9public static void main(String[] args) throws IOException, ClassNotFoundException {

10// Create a connection stream that knows how to read from a file.

11FileInputStream fileStream = new FileInputStream("three-dogs.ser");

12

13// Create a chain stream that knows how to read objects from an input stream.

14ObjectInputStream objectStream = new ObjectInputStream(fileStream);

15

16// Each time we call readObject(), we get the next object in the stream.

17Object one = objectStream.readObject();

18Object two = objectStream.readObject();

19Object three = objectStream.readObject();

20

21// If we want to use the objects as Dog objects, we need to cast them.

22Dog dog1 = (Dog) one;

23Dog dog2 = (Dog) two;

24Dog dog3 = (Dog) three;

25

26// Print the dog's information.

27System.out.println(dog1.getName() + " is hungry: " + dog1.isHungry());

28System.out.println(dog2.getName() + " is hungry: " + dog2.isHungry());

29System.out.println(dog3.getName() + " is hungry: " + dog3.isHungry());

30

31// Close the stream, thereby closing the file.

32objectStream.close();

33}

34}

35

Deserialisierung

AI

NEW

JAVA

RUN

STDIN

Input for the program (Optional)

Output:

Exception in thread "main" java.io.FileNotFoundException: three-dogs.ser (No such file or directory)
at java.base/java.io.FileInputStream.open0(Native Method)
at java.base/java.io.FileInputStream.open(FileInputStream.java:185)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:139)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:109)
at ObjectDeserializationDemo.main(ObjectDeserializationDemo.java:11)

Was während der Deserialisierung passiert

Deserialisierung

1. Das Objekt wird aus dem Stream **ausgelesen**.
 2. Die JVM ermittelt den **Typ der Klasse**.
 - Anhand der Informationen, die mit dem serialisierten Objekt gespeichert sind.
 3. Die JVM versucht, die **Klasse** des Objekts **zu finden und zu laden**.
 - Wenn die Klasse nicht gefunden/geladen werden kann, wirft die JVM eine Exception und die Deserialisierung schlägt fehl.
 4. Auf dem Heap wird Platz für ein neues Objekt reserviert. Der
- Konstruktor des Objekts wird jedoch **nicht** ausgeführt!
- Wenn doch, würde das Objekt als „new“ erzeugtes Objekt wieder auferstehen. Das wollen wir jedoch nicht. Stattdessen soll das Objekt in den Zustand zurückversetzt werden, in dem es war, als es serialisiert wurde.
5. Wenn das Objekt irgendwo in seinem **Vererbungsbaum** eine nicht serialisierbare Klasse hat, wird **der Standardkonstruktor für diese Klasse** zusammen mit allen Standardconstructoren für die übergeordneten Klassen ausgeführt
- (auch wenn diese serialisierbar sind).
- Die Konstruktorenkette kann nicht mehr gestoppt werden, sobald sie ihre Arbeit aufgenommen hat. Das hat zur Folge, dass der Zustand aller Superklassen, beginnend mit der ersten nicht serialisierbaren Klasse, neu initialisiert wird.
6. Die **Instanzvariablen** des Objekts erhalten die **Werte** zurück, die sie in ihrem **serialisierten Zustand** hatten.
 - **Transiente** Variablen erhalten für Objektreferenzen den Wert `null` und für elementare Typen die Standardwerte (`0`, `false`, ...).

! ? Quiz

🔥 Deserialisierung

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Während der Deserialisierung müssen der JVM die Klassen aller Objekte im Graphen zur Verfügung stehen.

Richtig!

Die Objekte werden (mit der Methode `readObject()`) in der Reihenfolge eingelesen, in der sie ursprünglich serialisiert wurden.

Richtig!

Der Rückgabotyp von `readObject()` ist
der des ursprünglich serialisierten Objekts.

Object! Deserialisierte Objekte müssen daher wieder in ihren ursprünglichen Typ zurückgecastet werden.

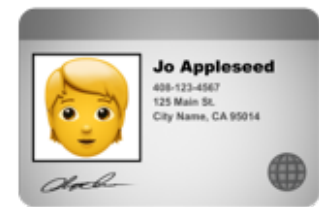
Statische Variablen werden nicht serialisiert. Es macht keinen Sinn, eine statische Variable als Teil eines bestimmten Objektzustands zu speichern, da alle Objekte dieses Typs den gleichen Wert verwenden – den von der Klasse definierten Wert.

Richtig!

 Welcher **eine** Gedanke zu
„ Deserialisierung“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren



Versions-ID

**Jetzt Seiten 556
und 557
wiederholen!**



Sie haben dafür 5 Minuten Zeit.

! Quiz

Versions-ID

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!

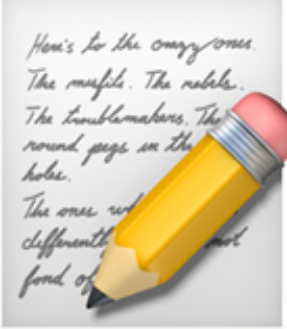


Wenn sich eine Klasse im Laufe der Zeit verändert, versehen Sie sie mit der Deklaration `static final long serialVersionUID`.

Richtig!

Diese Versions-ID sollte geändert werden,
wenn die serialisierten Werte der
Instanzvariablen geändert werden.

... wenn die serialisierten Variablen der Klasse geändert werden.

 Welcher **eine** Gedanke zu
„ Versions-ID“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Serialisierung und Datei-I/O

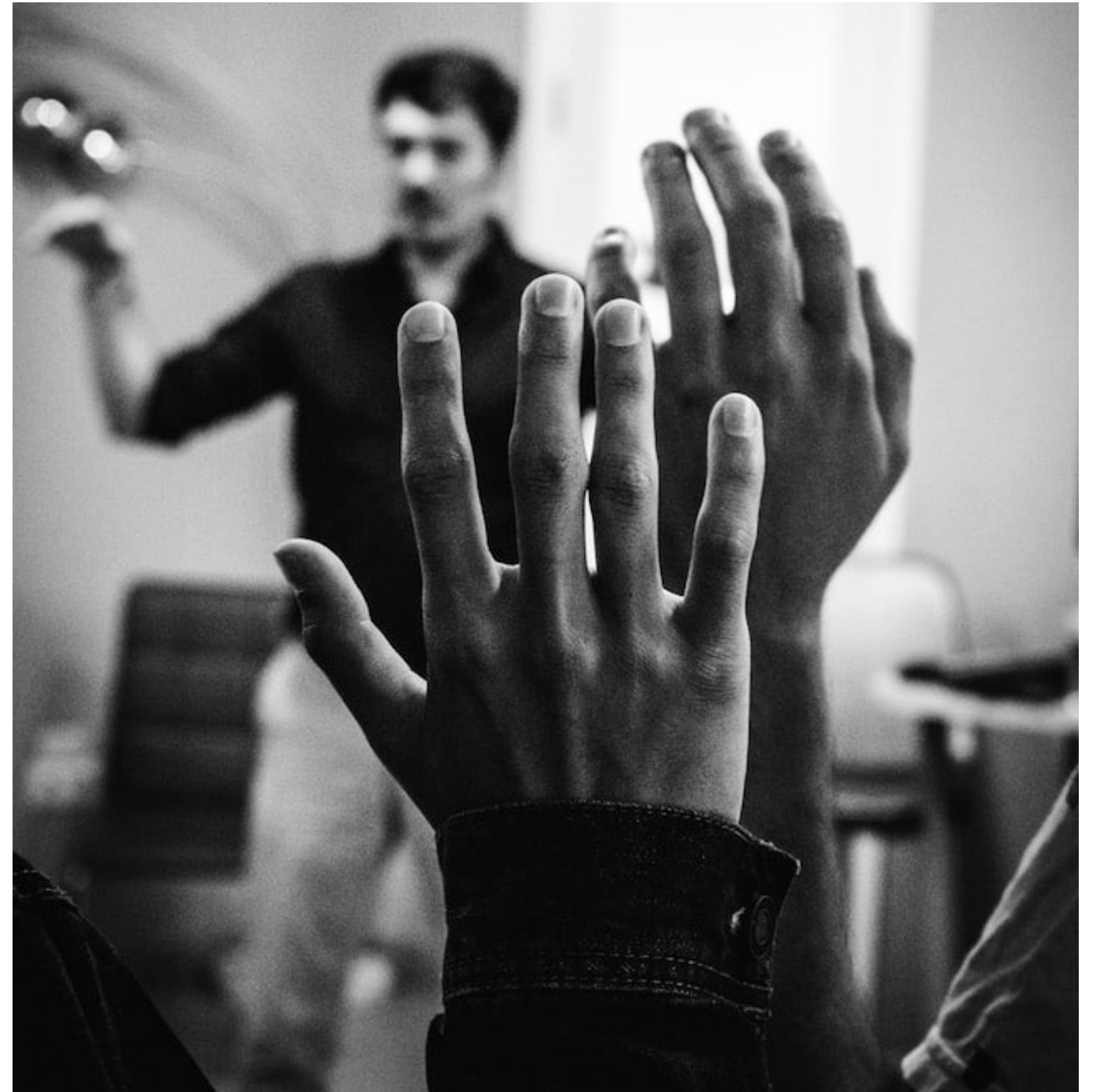
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Serialisierung und Datei-I/O?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Serialisierung und Datei-I/O



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

